

Member ID: _____

Time: _____

Rank: _____



C++ Programming

(335)

State 2025

PRODUCTION:

State_CPlusPlus

_____ (530 points)

Test Time: 90 minutes

GENERAL GUIDELINES:

Failure to adhere to any of the following rules will result in disqualification:

1. Member must hand in this test booklet and all printouts if any. Failure to do so will result in disqualification.
2. No equipment, supplies, or materials other than those specified for this event are allowed in the testing area. No previous BPA tests and/or sample tests (handwritten, photocopied, or keyed) are allowed in the testing area.
3. Electronic devices will be monitored according to ACT standards.

You will have ninety (90) minutes to complete your work.

Your name and/or school name should *not* appear on the work you submit for grading.

1. Create a folder on the flash drive provided using your contestant number as the name of the folder.
2. Submit your entire solution/project so that the graders may open your project to review the source code.
3. Ensure that the files required to run your program are present and will execute on the flash drive provided.

*Note that the flash drive letter may *not* be the same when the program is graded as it was when you created the program.

The graders will *not* alter your source code. Submissions that do *not* contain source code will *not* be graded.

Assumptions to make when taking this assessment:

- The goal of this assessment is to create a C++ console app that will allow the user to access a menu to generate a speeding ticket and to print the ticket information to the console.
- There is one file you need to program: State_CPlusPlus.cpp (NOTE: this file will need to access the file Names.txt file to create the records).
- No global variables are allowed

Development Standards:

- Your Code must use a consistent variable naming convention.
- All subroutines (if any), functions (if any), and methods (if any) must be documented with comments explaining the purpose of the method, the input parameters (if any), and the output (if any). Readability is a goal of good code.

Commenting for Source Code Review (see the rubric):

- Certain sections of your code will be graded. These gradable blocks of code can range from creating data structures, method algorithms, exception handling, and class construction.
- The grading rubric contains a section called Source Code Review: in this section are listed a description of all the graded programming concepts.
- Each gradable item must have a comment listed at its beginning, and the comment must be prefixed with the comment flag. The flag helps the graders easily locate the code to increase the effectiveness of grading.
- The flag will always use this naming convention: **SC#** (NOTE: the # symbol will be replaced with sequential numbering, i.e. **SC1**, **SC2**, **SC3**, etc.
- No explanation in the comment with the flag is required, only the comment flag; however, any information placed in the comment could help the grader better understand and avoid any costly errors.
- The comment flag needs to be placed in close proximity to the block of code it represents.
- If a comment flag is not present, you will not receive credit.
- In this example the Source Code Review has a gradable section of code for printing to the console (Remember these are non-related examples):
 - SC12: **WriteLine** method in the **Form1** class is printing the correct object _____ 10 pts
 - The user will place the code above the method call:

```
//SC12 printing a sarcastic comment  
Console.WriteLine("Get ready for some fun!");
```

C++ State Overview

You will be developing a console application inside of C++. The goal of this application is to read an exported database file that contains records of people who received speeding tickets, and to print out the records to the console for further analysis.

Premade CPP File

You will be given a C++ file that will contain only the preprocessor directives. There is no code supplied; however, you will be required to create certain elements with specific names of classes and other objects as part of the contest instructions. This will be explained in the Requirements section. You will also be given a text file called Names.txt which will contain all of the records your program will import and organize by assigning the field to the proper variables for object construction. These objects that are constructed will be used to print out to the console of all of the imported data. The data in the text file will be separated with comma delimiters.

Input/Output

When the program begins your program will automatically read the text file and organize the data based upon the order of the fields in the record that are separated by columns. Each line is a different record. Once the program is finished it will print out to the console. An image has been provided below so you can see what the output should look like. There is no user input or interaction with this program. It is fully automated. There is a total of 12 records that will be read and imported. The image to the right only shows the first 5.

A few details to notice on the output is that you were required to have the form header that says BPA spending ticket database at the top. The header in front of each record (record header) will be preceded by a series of asterisks an angle braces they'll have an incrementing number that must be coded into your solution. As you can see the record header increments by 1 value for each record as it's printed. To assist you is a given that there are 12 total records so you can use this knowledge to ensure that you are importing all of them properly. Additionally, you will be required to format the output of the records exactly as you see them in the image on this page. It should be noted that all of the fields are preceded by the field descriptor, for example "Name" is in front of the person's name; The exception to this rule will be the credit card number that will just appear on the same line as the Speeding Ticket Payment descriptor.

```
*****
**  BPA Speeding Ticket Database  **
*****

*****>>> 1 <<<*****
Name: John Doe
Car Information: Tesla Model S XYZ-789
Speeding Ticket Payment: $143.21 QWERTY1234567890

*****>>> 2 <<<*****
Name: Jane Smith
Car Information: Volkswagen Golf MNO-456
Speeding Ticket Payment: $77.50 ASDFGH0987654321

*****>>> 3 <<<*****
Name: Robert Johnson
Car Information: Subaru Outback JKL-012
Speeding Ticket Payment: $112.80 ZXCVCBN5678901234

*****>>> 4 <<<*****
Name: Sarah Brown
Car Information: BMW 3 Series RST-789
Speeding Ticket Payment: $50.25 UIOPJK6789012345

*****>>> 5 <<<*****
Name: Michael Taylor
Car Information: Audi A4 UVW-901
Speeding Ticket Payment: $99.99 YUIOPL3456789012
```

```
*****>>> 9 <<<*****
Name: Matthew Wilson
Car Information: Kia Sorento OPQ-345
Speeding Ticket Payment: $111.11 UIOPJK5678901234

*****>>> 10 <<<*****
Name: Olivia Anderson
Car Information: Chevrolet Equinox YUI-678
Speeding Ticket Payment: $150.00 ASDFGH3456789012

*****>>> 11 <<<*****
Name: Ethan Garcia
Car Information: GMC Sierra XYZ-987
Speeding Ticket Payment: $65.40 QWERTY5678901234

*****>>> 12 <<<*****
Name: Ava Martinez
Car Information: Jeep Wrangler JKL-789
Speeding Ticket Payment: $85.75 UIOPLK6789012345

*****
```

The printout to your left shows the bottom half of the print out. Pay special attention that below the 12 record is a form footer that is a line made of Asterix.

Text File: Names.txt

Below you will find an image of what the text file looks like. It is organized in the following manner: first name, last name, car make, car model, license plate number, ticket amount, credit card number. Do not be concerned about what the values are and only be concerned about the order and that they are separated by commas. This is the relative file location for the Names.txt file: CPP_State\CPP_State

```
John, Doe, Tesla, Model S, XYZ-789, 143.21, QWERTY1234567890
Jane, Smith, Volkswagen, Golf, MNO-456, 77.50, ASDFGH0987654321
Robert, Johnson, Subaru, Outback, JKL-012, 112.80, ZXCVCN5678901234
Sarah, Brown, BMW, 3 Series, RST-789, 50.25, UIOPJK6789012345
Michael, Taylor, Audi, A4, UVW-901, 99.99, YUIOPL3456789012
Emily, White, Ford, Mustang, NOP-890, 75.80, ASDFGH1234567890
Daniel, Miller, Nissan, Altima, GHI-234, 121.33, QWERTY0987654321
Rachel, Davis, Hyundai, Sonata, STU-567, 88.70, ZXCVMN2345678901
Matthew, Wilson, Kia, Sorento, OPQ-345, 111.11, UIOPJK5678901234
Olivia, Anderson, Chevrolet, Equinox, YUI-678, 150.00, ASDFGH3456789012
Ethan, Garcia, GMC, Sierra, XYZ-987, 65.40, QWERTY5678901234
Ava, Martinez, Jeep, Wrangler, JKL-789, 85.75, UIOPLK6789012345
```

Error Handling

The only error handling your program should manage is in the event that it cannot read the file or it cannot find the file. You'll be required to put this in your code and it will be part of the points on the rubric. However, your focus should be mainly on making the software work and reading the file as you will receive more points for its functionality than error handling.

Requirements/Explanations

1. You must use the provided Visual Studio project called “**State_CPlusPlus**” in the C++ State Student folder.
2. This project contains a “**State_CPlusPlus.cpp**” that has been already created; you will also receive a text file called Names.txt (this file contains the database records that your program will read, organize, and print to the console).
3. No additional files can be added to the project.
4. The **State_CPlusPlus.cpp** file does not contain any code as you will need to create the majority of all classes, variables, methods, and control logic. Listed below are the required elements for your program:
 - a. **class Name**: this will be the object class for the first and last name to be stored.
 - b. **class Name** class method **getFullName()**: this method will return the full name
 - c. **class CarInfo**: this will be the object class that stores the information for the car make, the car model, and the license plate.
 - d. **class CarInfo** method **getInfo()**: this method will return the car make, car model, and license plate.
class Payment: this will be the object class that will store how much is paid for the ticket and the credit card number
 - e. **class Payment** class method **getPaymentInfo()**: this method will return the payment information as a string there has been formatted for currency. The credit card number needs to be returned from this method as well.
 - f. **class Customer** this is the class object that will store the Name object, the CarInfo object, and the Payment object.
 - g. **class Customer** class method **printInfo()**: this method we'll have the responsibility of printing out all of the information from the objects stored within the customer object. This is a good place for the field descriptors to be organized also.
 - h. **class TicketDriver** this is the driver class that has a multitude of functions within it. One of the first is to provide and error handling ability in case there is problems with opening the file. Next this class will be reading the text file provided and organizing the information to be stored in proper variables that can be used to construct the other class objects that were previously discussed. It is also the location for all of the objects to be stored. The actual printing to the console will take place in this class.
 - i. **class TicketDriver** class method **run()**: this method will be the central location where all actions for this class take place. The two main actions for this class is reading the data and printing to the console.
 - j. **class TicketDriver** class method **readDataFromFile()**: this method will be responsible for opening the file in reading the file line by line in separating the data based upon the delimiter that is used to separate the data fields in the text file.
 - k. **class TicketDriver** class method **printRecords()**: this method will be responsible for taking the organized data that has been stored in the customer object and providing any formatting for the console output that you need to create.

RUBRIC

Solution and Project (There is NO partial credit) (NOTE: UC represents uppercase and LC represents lowercase)		
The project file is present on the flash drive in a single folder with your contest ID		10 points
Program Execution (If the program does not execute, then the remaining items in this section receive a score of zero)		
When launched the program starts with no errors		20 points
The beginning of the print out has the “BPA Speeding Ticket Database” form header; the format matches the sample provided in the packet with less than two deviations		10 points
There are 12 total records that have printed out with the record header the increments by 1 for each print out.		10 points
All 12 record headers are identical accept for the incrementing number and the format matches the example provided with less than two deviations		10 points
Every record includes the filled descriptors exactly as shown in the sample output.		10 points
The data and the records is printed in the exact the order in which it is listed in the “Names.txt” file		50 points
The format for all the records matches the sample output exactly. This includes all spacing and the order of information.		50 points
The form footer is included after the 12th record is printed it matches the sample output exactly.		10 points
Subtotal		/180 Points

Source Code Review (There is NO partial credit) No credit will be given for missing flagged comments Code segments must work to receive full credit		
A comment containing the contestant number is present at the top of the CPP_State.cpp file.		10 points
SC1: Flag Comment Present		5 points
SC1: class Name the constructor handles parameters for the first and last name		15 points
SC2: Flag Comment Present		5 points
SC2: class Name method getFullName() returns the first and last name together.		10 points
SC3: Flag Comment Present		5 points
SC3: class Name has no public variables		10 points
SC4: Flag Comment Present		5 points
SC4: class CarInfo the constructor handles parameters for the car make, car model, and license plate.		15 points
SC5: Flag Comment Present		5 points
SC5: class CarInfo method getInfo() returns the car make, car model and license plate together.		10 points
SC6: Flag Comment Present		5 points
SC6: class CarInfo class has no public variables		10 points
SC7: Flag Comment Present		5 points
SC7: class Payment the constructor handles parameters for the ticket payment amount and credit card number.		15 points
SC8: Flag Comment Present		5 points
SC8: class Payment method returns the formatted payment value for USD using stringstream and setprecision and it also returns the credit card number.		15 points
SC9: Flag Comment Present		5 points
SC9: class Payment class uses no public variables		10 points
SC10: Flag Comment Present		5 points
SC10: class Customer the constructor handles parameters for the following objects: Name, CarInfo, and Payment.		10 points
SC11: Flag Comment Present		5 points
SC11: class Customer method creates the field descriptors for the three objects and gets the proper information from each object by using the object getter methods.		10 points
SC12: Flag Comment Present		5 points
SC12: class Customer class has no public variables.		10 points
SC13: Flag Comment Present		5 points

SC13: class Customer class method run() controls the program by initiating the reading of the files and printing the records using readDataFromFile() and printRecords().		10 points
SC14: Flag Comment Present		5 points
SC14: TicketDriver class method readDataFromFile() finds Names.txt and gives error message if the file cannot open properly		15 points
SC15: Flag Comment Present		5 points
SC15: TicketDriver class method readDataFromFile() uses stringstream that reads each line and looks for commas and assigns values to all of the variables using a loop structure.		40 points
SC16: Flag Comment Present		5 points
SC16: SC15: TicketDriver class method readDataFromFile() uses error handling for parsing error, AND properly closes file once completed.		20 points
SC17-15: Flag Comment Present		5 points
SC17: SC15: TicketDriver class method readDataFromFile() properly closes file once completed.		20 points
SC18: Flag Comment Present		5 points
SC18: TicketDriver class method void printTickets(): method uses a loop to print all available tickets.		20 points
SC19: Flag Comment Present		5 points
SC19: TicketDriver class method void printTickets(): method uses a loop to update the counter on the record header (remember it will count from 1 to 12).		10 points
Subtotal		/380 Points
Total Points		/ Points